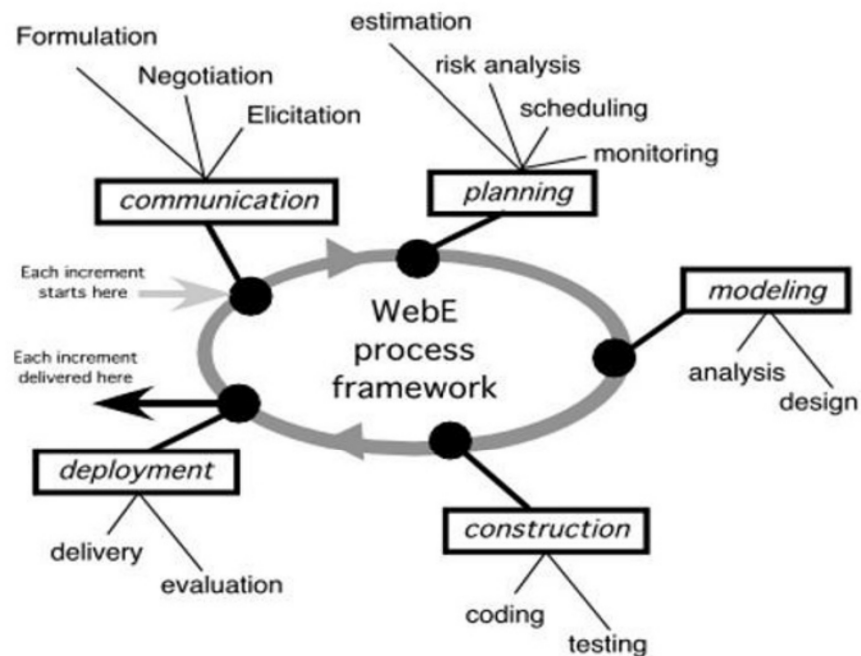


Lecture 11 Notes: Testing



Testing WebApps

- Testing is the process of **exercising a WebApp to find** (and ultimately correct) **errors**.
- Tests must be designed to uncover errors in **WebApps** that are **implemented in**:
 - **different operating systems**
 - **browsers** [or other interface devices such as set-top boxes, personal digital assistants (PDAs), and mobile phones]
 - **hardware** platforms
 - communications **protocols**
 - **“backroom” applications**

The “Dimensions” of Quality

- Reviews and testing examine one or more of the following quality dimensions:
 - **Content** is evaluated at both a syntactic and semantic level. At the syntactic level, spelling, punctuation, and grammar are assessed for text-based documents. At a semantic level, correctness (of information presented), consistency (across the entire content object and related objects), and lack of ambiguity are all assessed.
 - **Function** is tested to uncover errors that indicate lack of conformance to stakeholder requirements. Each WebApp function is assessed for correctness, instability, and general conformance to appropriate implementation standards (e.g., Java or XML language standards).

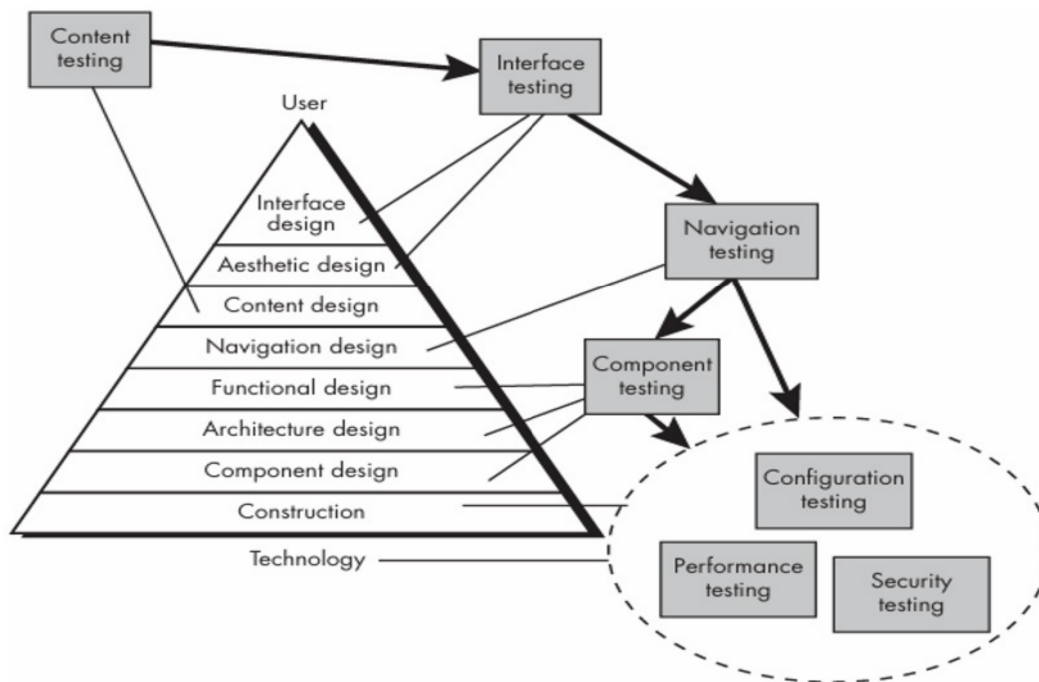
- **Structure** is assessed to ensure that it properly delivers WebApp content and function, is extensible, and can be supported as new content or functionality is added.
- **Usability** is tested to ensure that each category of user is supported by the interface and can learn and apply all required navigation syntax and semantics.
- **Navigability** is tested to ensure that all navigation syntax and semantics are exercised to uncover any navigation errors (e.g., dead links, improper links, erroneous links).
- **Performance** is tested under a variety of operating conditions, configurations, and loading to ensure that the system is responsive to user interaction and handles extreme loading without unacceptable operational degradation.
- **Compatibility** is tested by executing the WebApp in a variety of different host configurations on both the client and server sides. The intent is to find errors that are specific to a unique host configuration.
- **Interoperability** is tested to ensure that the WebApp properly interfaces with other applications and/or databases.
- **Security** is tested by assessing potential vulnerabilities and attempting to exploit each. Any successful penetration attempt is deemed a security failure.

Testing Strategy

1. The content model for the WebApp is reviewed to uncover errors.
2. The interface model is reviewed to ensure that all use cases have been accommodated.
3. The design model for the WebApp is reviewed to uncover navigation errors.
4. The user interface is tested to uncover errors in presentation and/or navigation mechanics.
5. Selected functional components are unit tested.
6. Navigation throughout the architecture is tested.
7. The WebApp is implemented in a variety of different environmental configurations and is tested for compatibility with each configuration.
8. Security tests are conducted to exploit vulnerabilities in the WebApp or within its environment.
9. Performance tests are conducted.
10. The WebApp is tested by a controlled and monitored population of end users. The results of their interaction with the system are evaluated for content and navigation errors, usability concerns, compatibility concerns, and WebApp reliability and performance.

The Testing Process

Note that: In the following figure, as the testing flow proceeds from left to right and top to bottom, user-visible elements of the WebApp design (top elements of the pyramid) are tested first, followed by infrastructure design elements.



1- Content Testing

- Content testing combines both reviews and the generation of executable test cases.
 - **Reviews** are applied to uncover semantic errors in content.
 - **Executable testing** is used to uncover content errors that can be traced to dynamically derived content that is driven by data acquired from one or more databases.
- Content testing has three important objectives:
 - **to uncover syntactic errors** (e.g., typos, grammar mistakes) in text-based documents, graphical representations, and other media,
 - **to uncover semantic errors** (i.e., errors in the accuracy or completeness of information) in any content object presented as navigation occurs, and
 - **to find errors in the organization or structure of content** that is presented to the end user.

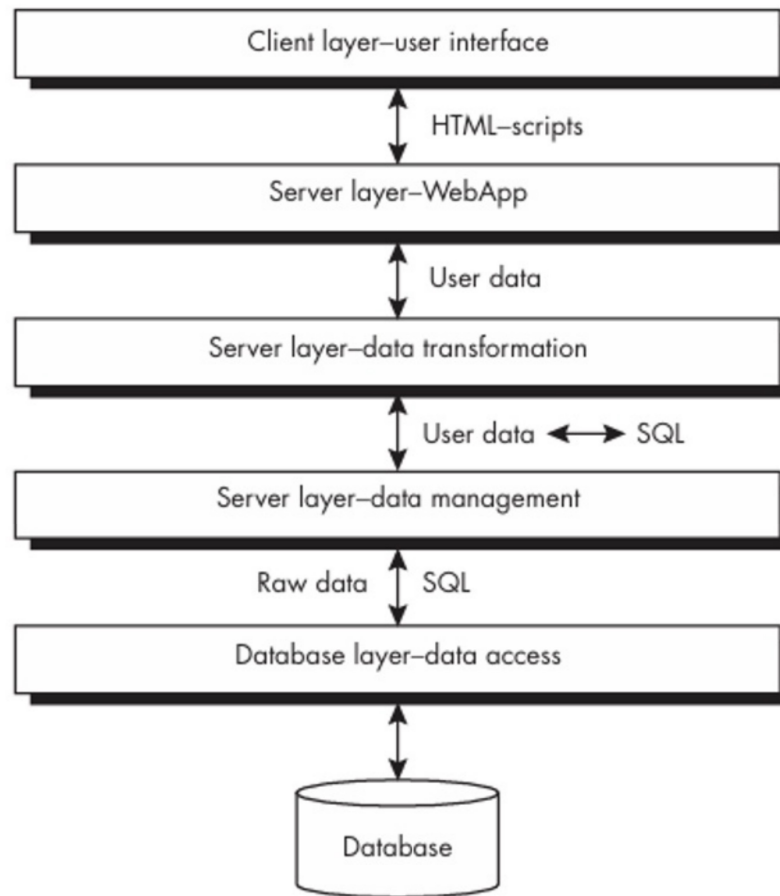
Content Testing – Checklist

- ✓ Is the information **up to date** and factually **accurate**?
- ✓ Is the information **concise** and to the point?
- ✓ Is the **layout** of the content object easy for the user to understand?

- ✓ Can **information embedded** within a content object be found easily?
- ✓ Have proper **references** been provided for all information derived from other sources?
- ✓ Is the information presented **consistent** internally and consistent with information presented in other content objects?
- ✓ Can the content be interpreted as being **offensive or misleading**, or does it open the door to **litigation**?
- ✓ Does the content infringe on existing **copyrights** or trademarks?
- ✓ Does the content contain internal links that supplement existing content? Are the **links correct**?
- ✓ Does the **aesthetic** style of the content conflict with the aesthetic style of the interface?

Content Testing – Dynamic Content

- When content is created dynamically using information maintained within a database, the following issues are considered:
 1. The original client-side request for information is rarely presented in the form [e.g., structured query language (SQL)] that can be input into a database management system (DBMS).
 2. The database may be remote to the server that houses the WebApp.
 - *What happens if the WebApp is accessible, but the database is not?*
 3. Raw data acquired from the database must be transmitted to the WebApp server and properly formatted for subsequent transmittal to the client.
 4. The dynamic content object(s) must be transmitted to the client in a form that can be displayed to the end user.
- Considering the previous four factors, test case design methods should be applied for each of the “layers of interaction” noted in the following figure. Testing should ensure that:
 1. Valid information is passed between the client and server from the interface layer.
 2. WebApp processes scripts correctly and properly extracts or formats user data.
 3. User data is passed correctly to a server-side data transformation function that formats appropriate queries (e.g., SQL).
 4. Queries are passed to the data management layer that communicates with database access routines (potentially located on another machine).



2- User Interface Testing

- Verification and validation of a WebApp user interface occurs at three distinct points in the WebE process.
 - **During communication** and modeling, the interface model is reviewed to ensure that it conforms to customer requirements and to other elements of the analysis model.
 - **During design**, the interface design model is reviewed to ensure that generic quality criteria established for all user interfaces have been achieved and that application-specific interface design issues have been properly addressed.
 - **During testing**, the focus shifts to the execution of application-specific aspects of user interaction as they are manifested by interface syntax and semantics. In addition, testing provides a final assessment of usability.

UI Testing Strategy

- Interface features are tested to ensure that design rules, aesthetics, and related visual content are available to the user without error.

- Individual interface mechanisms are tested in a manner that is analogous to unit testing.
- Each interface mechanism is tested within the context of a use case or navigation pathway for a specific user category.
- The complete interface is tested against selected use cases and navigation pathways to uncover errors in the semantics of the interface.
- The interface is tested within a variety of environments (e.g., operating systems, browsers) to ensure that it will be compatible.

UI Testing specific elements

- When a user interacts with a WebApp, the interaction occurs through one or more interface mechanisms. Each mechanism must be tested:
 - **Links.** Navigation mechanisms that link the user to some other content object or function.
 - **Forms.** A structured document containing blank fields that are filled in by the user.
 - **Client-side scripting.** A list of programmed commands in a scripting language (e.g., JavaScript) that handle information input via forms or other user interactions.
 - **Dynamic HTML.** Provides access to content objects that are manipulated on the client side using scripting or cascading style sheets (CSSs).
 - **Client-side pop-up windows.** Small windows that pop up without user interaction.
 - **Server-side scripts.** Black-box tests are conducted with an emphasis on data integrity and script processing once validated data has been received. In addition, performance testing can be conducted.
 - **Streaming and push content.** *Streaming content* is encountered when material (usually audio or video) is downloaded in a manner that allows it to be displayed while it is still being downloaded (rather than having to wait for the entire content to be downloaded). *Push content* is encountered when content objects are downloaded automatically from the server side rather than waiting for a request from the client side. Both streaming and push content present testing challenges.
 - **Cookies.** A block of data sent by the server and stored by a browser because of a specific user interaction. The content of the data is WebApp-specific (e.g., user identification data or a list of items that have been selected for purchase by the user).
 - **Application-specific interface mechanisms.** Include one or more “macro” interface mechanisms such as a shopping cart, credit card processing, or a shipping cost calculator.

Usability Testing

- Similar to interface semantics testing in the sense that it evaluates:
 - The degree to which users can interact effectively with the WebApp
 - The degree to which the WebApp guides users' actions provides meaningful feedback and enforces a consistent interaction approach.
- Determines the degree to which the WebApp interface makes the user's life easy

3- Navigation Testing

- Each of the following navigation mechanisms should be tested:
 - **Navigation links.** These mechanisms include internal links within the WebApp, external links to other WebApps, and anchors within a specific Web page.
 - **Redirects.** These links come into play when a user requests a nonexistent URL or selects a link whose destination has been removed or whose name has changed.
 - **Bookmarks.** Although bookmarks are a browser function, the WebApp should be tested to ensure that a meaningful page title can be extracted as the bookmark is created and that dynamic pages are bookmarked appropriately.
 - **Frames and framesets.** Each frame contains the content of a specific Web page; a frameset contains multiple frames and enables the display of multiple Web pages at the same time.
 - **Site maps.** A site map provides a complete table of contents for all Web pages.
 - **Internal search engines.** An internal (local) search engine allows the user to perform a key word search within the WebApp to find the content needed.

Navigation Semantics

- As navigation design is conducted, you create “a set of information and related navigation structures that collaborate in the fulfillment of a subset of related user requirements”.
- These are sometimes referred to as *navigation semantic units* (NSUs) and are defined by a set of navigation paths (called “ways of navigating”) that connect navigation nodes (e.g., Web pages, content objects, or functionality).
- Taken as a whole, each NSU allows a user to achieve specific requirements defined by one or more use cases for a user category.
- **Navigation testing exercises each NSU to ensure that these requirements can be achieved.**

4- Component-Level Testing

- *Component-level testing*, also called *function testing*, focuses on a set of tests that attempt to uncover errors in WebApp functions
- Applies the following test-case design methods:
 - **Equivalence partitioning:** Test cases for each class of input are derived and executed, while other classes of input are held constant. Ex.: an e-commerce application implements a function that computes shipping charges. Among shipping information is the user's postal code. Test cases are designed to uncover errors in postal code processing by specifying postal code values that might uncover different classes of errors (e.g., an incomplete postal code, a correct postal code, a nonexistent postal code, an erroneous postal code format).
 - **Boundary value analysis:** Forms data are tested at their boundaries. Ex.: a shipping calculation function requests the maximum number of days required for product delivery. A minimum of 2 days and a maximum of 14 are noted on the form. However, boundary value tests might input values of 0, 1, 2, 13, 14, and 15 to determine how the function (and associated error processing) reacts to data at and outside the boundaries of valid input.
 - **Path testing:** is a white-box testing technique that focuses on designing test cases to execute all possible linearly independent paths within a program or module's control structure.
 - ⇒ **White-box testing** examines internal code structure for logic and security, requiring programming knowledge (ex. unit testing).
 - ⇒ **Black-box testing** checks external functionality from a user's perspective, needing no code knowledge (ex. user acceptance testing).

Selecting Components to Test

- Which functionality in the Web site is most critical to its purpose?
- Which areas of the site require the heaviest database interaction?
- What areas of the site will be the most popular?
- What aspects of the site have the highest security risks?

5- Configuration Testing

- Configuration variability and instability are important factors that make Web engineering a challenge.
 - Hardware, operating system(s), browsers, storage capacity, network communication speeds, and a variety of other client-side factors are difficult to predict for each user.

- The job of configuration testing is to test a set of probable client-side and server-side configurations to ensure that the user experience will be the same on all of them and to isolate errors that may be specific to a particular configuration.

Configuration Testing Strategy

- **Server-side:** Configuration **test** cases are designed to verify that the projected **server configuration** [i.e., WebApp server, database server, operating system(s), firewall software, concurrent applications] can support the WebApp without error.
- **Client-side:** On the client side, configuration **tests focus** more heavily **on WebApp compatibility** with configurations that contain one or more permutations of the following components:
 - **Hardware.** CPU, memory, storage, and printing devices
 - **Operating systems.** Linux, Macintosh OS, Microsoft Windows, a mobile-based OS
 - **Browser software.** FireFox, Internet Explorer, Safari, Mozilla/Netscape, Opera, and others
 - **User interface components.** Active X, Java applets, and others
 - **Plug-ins.** QuickTime, RealPlayer, and many others
 - **Connectivity.** Cable, DSL, regular modem, industry-grade connectivity (e.g., T1 lines)

6&7- Security and Performance Testing

- Security and performance testing address the three distinct elements of the WebApp infrastructure
 - the server-side environment that provides the gateway to Internet users
 - the network communication pathway between the server and the client machine
 - the client-side environment that provides the end user with a direct interface to the WebApp.
- **Security testing** focuses on unauthorized access to WebApp content and functionality along with other systems that cooperate with the WebApp on the server side.
- **Performance testing** focuses on the operating characteristics of the WebApp and on whether those operating characteristics meet the needs of end users.

6- Security Testing

- One or more of the following security elements is implemented:
 - **Firewalls.** A filtering mechanism that is a combination of hardware and software examines each incoming packet of information to ensure that it is coming from a legitimate source, blocking any data that is suspected.

- **Authentication.** A verification mechanism that validates the identity of all clients and servers, allowing communication to occur only when both sides are verified.
- **Encryption.** An encoding mechanism that protects sensitive data by modifying it in a way that makes it impossible to read by those with malicious intent. Encryption is strengthened by using *digital certificates* that allow the client to verify the destination to which the data is transmitted.
- **Authorization.** A filtering mechanism that allows access to the client or server environment only by those individuals with appropriate authorization codes (e.g., user ID and password).
- Security tests should be designed to probe each of these security technologies to uncover security holes that can be exploited by those with malicious intent.

7- Performance Testing

- Objectives:
 - Does the server response time degrade to a point where it is noticeable and unacceptable?
 - At what point (in terms of users, transactions, or data loading) does performance become unacceptable?
 - What system components are responsible for performance degradation?
 - What is the average response time for users under a variety of loading conditions?
 - Does performance degradation have an impact on system security?
 - Is WebApp reliability or accuracy affected as the load on the system grows?
 - What happens when loads that are greater than maximum server capacity are applied?
 - What is the impact of poor performance on company revenues?
- **Load testing** determines how the WebApp and its server-side environment will respond to various loading conditions.
- **Stress testing** is a continuation of load testing, but in this instance the variables, *N*, *T*, and *D* are forced to meet and then exceed operational limits.
 - **N:** the number of concurrent users.
 - **T:** the number of online transactions per unit of time.
 - **D:** the data load processed by the server per transaction.
- Load testing can also be used to assess recommended connection speeds for users of the WebApp.
- **Overall throughput (P)** is computed in the following manner: $P = N * T * D$.

- **Ex.:** consider a popular sports news site. At any given time, 4000 con current users submit a request (a transaction T) once every 30 seconds on average. Each transaction requires the WebApp to download a news article that averages 12 Kbytes in length.
 - Therefore, $N = 4000$ users, $T = 0.033$ transactions per second per user, $D = 12$ Kbyte per transaction.
 - Throughput can be calculated as: $P = 4000 * 0.033 * 12 \approx 1600$ Kbyte/s
 - The network connection for the server would therefore have to support this average data rate and should be tested to ensure that it does.